X “For You” 推荐系统

技术架构与算法原理

——基于官方开源代码仓库的技术解析*

辛知航

江南大学数学与数据科学学院

摘要

X（原 Twitter）的 “For You” 信息流是全球最大规模的实时推荐系统之一。本文基于 X 开源的推荐算法代码仓库，系统性地分析了该系统的整体架构、核心算法流程、关键模型设计以及数学原理。系统采用经典的多阶段推荐架构——召回（Retrieval）、精排（Ranking）与重排（Re-ranking）——但在每个阶段均做出了独特的工程与算法决策。在召回阶段，系统融合了基于社交图谱的实时内网召回（Thunder）与基于双塔模型的外网向量检索（Phoenix Retrieval）；在排序阶段，系统基于 Grok 大语言模型衍生的 Transformer 架构，通过 “候选隔离注意力”（Candidate Isolation Attention）机制实现了对多种用户行为概率的联合预测；在重排阶段，通过加权融合、作者多样性衰减及可见性过滤等策略，生成最终的个性化信息流。

*https://github.com/xai-org/x-algorithm

目录

1 引言 **4**

2 系统整体架构 **4**

2.1 架构总览 4

2.2 数据流 4

2.3 离线与在线部分划分 5

3 核心算法流程 **6**

3.1 内网召回：Thunder 6

3.1.1 存储结构 6

3.1.2 回复过滤逻辑 6

3.1.3 自动过期与删除 6

3.2 外网召回：Phoenix Retrieval 6

3.2.1 模型架构 7

3.2.2 检索流程 7

3.3 精排：Phoenix Ranking 7

3.3.1 模型输入 7

3.3.2 候选隔离注意力 8

3.3.3 多行为预测 8

3.4 重排阶段 8

4 关键模型与算法 **8**

4.1 用户建模 8

4.1.1 显式用户特征 8

4.1.2 用户行为序列（UAS） 8

4.1.3 多哈希用户嵌入 9

4.2 内容建模 9

4.2.1 历史序列嵌入 9

4.2.2 行为嵌入 9

4.2.3 候选帖子嵌入 9

4.3 排序模型：Phoenix Transformer 9

4.3.1 RMS 层归一化 10

4.3.2 旋转位置编码（RoPE） 10

4.3.3 分组查询注意力（GQA） 10

4.3.4 SwiGLU 前馈网络 10

4.3.5 注意力 Logit 裁剪 11

4.3.6 Decoder Layer 结构 11

4.4 特征工程方法 11

4.5 过滤策略体系 11

5 数学原理与公式 **11**
5.1 双塔模型与向量检索 11
5.2 候选隔离注意力机制 12
5.3 多行为联合预测 13
5.4 加权评分公式 13
5.4.1 分数归一化 14
5.4.2 视频权重条件 14
5.5 作者多样性衰减 14
5.6 外网内容调整 15
5.7 哈希嵌入与冲突缓解 15
5.8 布隆过滤器在去重中的应用 16

6 管道框架设计 **16**
6.1 Trait 抽象 16
6.2 设计原则 16

7 设计哲学与讨论 **17**
7.1 去特征工程化 17
7.2 候选隔离的工程意义 17
7.3 正负信号的联合建模 17
7.4 内外网融合策略 17

8 总结 **18**

A 符号表 **18**

1 引言

社交媒体平台的推荐系统承担着在海量内容中为每位用户筛选出最相关、最有价值信息的核心任务。X 平台的 “For You” 信息流每天为数亿用户提供个性化的内容推荐，其技术实现代表了工业级推荐系统的前沿实践。

本文基于 X 开源的推荐算法代码仓库[1]，从以下四个维度展开系统性分析：

(1) **系统整体架构**：数据流、模块职责、离线与在线部分划分；

(2) **核心算法流程**：从用户请求到结果返回的完整推荐流程；

(3) **关键模型与策略**：用户建模、内容建模、排序模型、特征工程方法；

(4) **数学原理与公式**：对关键算法给出数学表达与必要推导。

2 系统整体架构

2.1 架构总览

X 的 For You 推荐系统由四个核心模块组成，如 table 1所示。系统整体遵循 “管道式”（Pipeline）设计，数据从用户请求出发，经过多个阶段的处理，最终返回排序后的帖子列表。

表 1: 系统核心模块

模块	代码位置	核心职责
Home Mixer	`home-mixer/`	编排层：组装并调度整个推荐管道
Thunder	`thunder/`	内网召回：实时内存帖子存储与快速检索
Phoenix	`phoenix/`	ML 模型：外网召回（双塔）与精排（Transformer）
Candidate Pipeline	`candidate-pipeline/`	通用框架：提供可组合的推荐管道抽象

2.2 数据流

一次完整的 For You 请求经历如 fig. 1所示的数据流转。

Stage 1: 查询水合。当客户端发起请求时，系统首先并行获取用户的上下文信息。用户行为序列（User Action Sequence, UAS）记录了用户近期在平台上的所有交互行为——点赞、转发、回复、点击等——并经过聚合和过滤后压缩为一个定长序列，作为排序模型的核心输入。用户特征（User Features）包括关注列表、屏蔽列表、静音关键词等，用于后续过滤。

Stage 2: 候选源召回。系统并行地从两个来源获取候选帖子：Thunder 提供来自用户关注账户的最新帖子（内网候选），Phoenix Retrieval 通过双塔模型从全局语料库中检索出语义相关的帖子（外网候选）。两类候选合并后进入后续处理。

[1]代码仓库包含 `home-mixer`、`thunder`、`phoenix`、`candidate-pipeline` 四个核心模块。

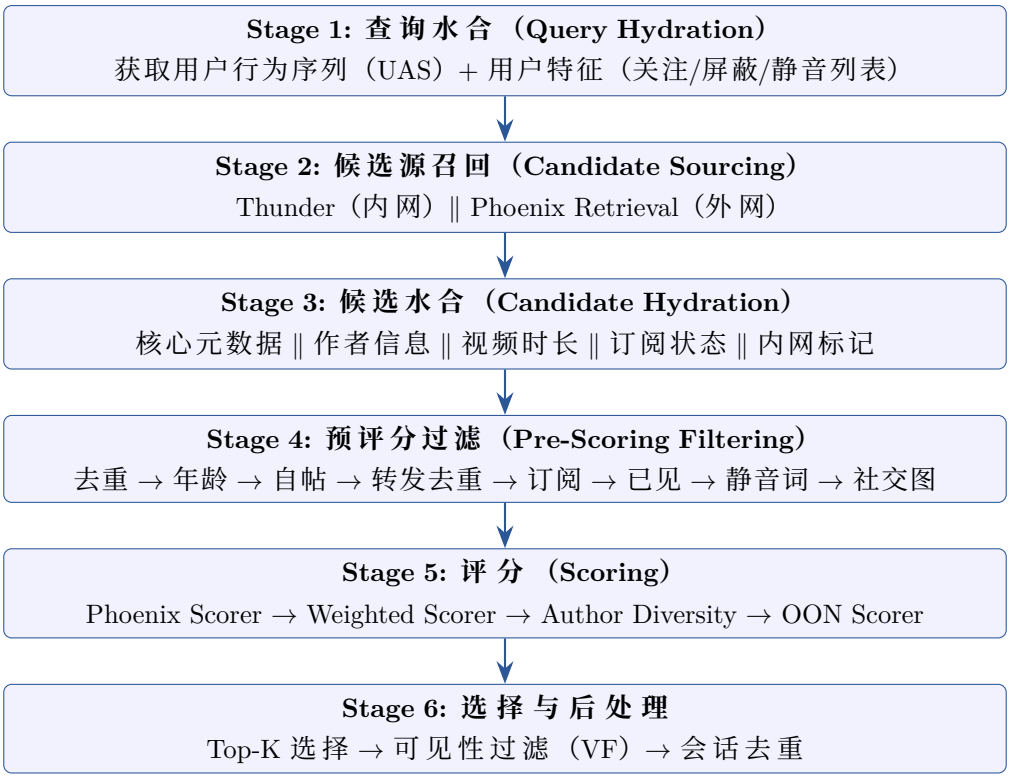

图 1: 推荐管道的六阶段数据流

Stage 3: 候选水合。对每个候选帖子，系统并行地获取核心元数据（文本、作者 ID、转发关系）、作者信息（粉丝数、用户名）、视频时长、订阅状态以及是否属于内网等信息。

Stage 4: 预评分过滤。系统依次执行一系列过滤器，移除不适合展示的候选。过滤器顺序执行，每个过滤器将候选集划分为 “保留” 和 “移除” 两个子集。

Stage 5: 评分。多个评分器依次运行：首先由 Phoenix Scorer 调用 Transformer 模型获取多行为概率预测，然后由 Weighted Scorer 将这些概率加权融合为综合分数，再由 Author Diversity Scorer 施加作者多样性衰减，最后由 OON Scorer 调整外网内容分数。

Stage 6: 选择与后处理。按最终分数降序排列，选取前 K 个候选。之后进行可见性过滤和会话去重，最终返回给客户端。

2.3 离线与在线部分划分

推荐系统清晰地分为离线与在线两个部分：

- **离线部分**：模型训练（Phoenix Transformer 的参数学习）、用户行为序列的持续聚合与存储、全局语料库的向量索引构建（候选塔离线计算所有帖子的嵌入向量）。
- **在线部分**：实时的用户请求处理管道、Thunder 的实时帖子摄入（通过 Kafka 消费发帖/删帖事件）、Phoenix 的在线推理（用户塔实时编码 + ANN 检索 + Transformer 排序）。

这种划分的核心优势在于：离线部分承担计算密集的模型训练与索引构建，在线部分则聚焦于低延迟的检索与推理，两者通过嵌入向量和模型参数实现解耦。

3 核心算法流程

3.1 内网召回：Thunder

Thunder 是一个基于内存的实时帖子存储系统，通过 Kafka 消费所有帖子的创建与删除事件，维护按作者分组的帖子时间线。当用户请求到达时，Thunder 根据用户的关注列表，以亚毫秒级延迟检索出这些作者的近期帖子。

3.1.1 存储结构

Thunder 的存储结构将帖子分为三类时间线，如 table 2所示。

表 2: Thunder 帖子存储结构

存储	内容	采样上限
原创时间线	非回复、非转发的原创帖子	每作者 N_{orig} 条
二级时间线	回复与转发	每作者 N_{sec} 条
视频时间线	含合格视频的帖子	每作者 N_{video} 条

每类时间线按作者维护独立的双端队列（VecDeque），按创建时间排序。这种分离设计允许系统对不同类型的内容施加不同的采样策略。

3.1.2 回复过滤逻辑

对于二级帖子（回复与转发），Thunder 执行额外的过滤：只保留对**原始帖子**（非回复链深处）的直接回复，或者对**关注用户帖子**的回复。具体而言，设关注用户集合为 $\mathcal{F}$，帖子 p 的回复目标帖子为 p_{reply}，则保留条件为：

$$\text{keep}(p) = \neg p_{\text{reply}}.\text{is_reply} \ \vee \ \big(\text{author}(p_{\text{reply}}) \in \mathcal{F} \ \wedge \ p_{\text{reply}}.\text{in_reply_to} = p_{\text{conv_root}}\big) \tag{1}$$

这一策略有效地控制了回复链的深度，避免向用户展示过多与其无直接关系的对话碎片。

3.1.3 自动过期与删除

Thunder 实现了后台自动过期机制——定期扫描所有时间线，移除超过保留期 T_{ret} 的帖子并释放内存。被删除的帖子记录在独立的集合 $\mathcal{D}$ 中，在检索时被排除。

3.2 外网召回：Phoenix Retrieval

Phoenix Retrieval 采用**双塔模型**（Two-Tower Model）从全局语料库中检索出与用户兴趣语义相关的帖子。

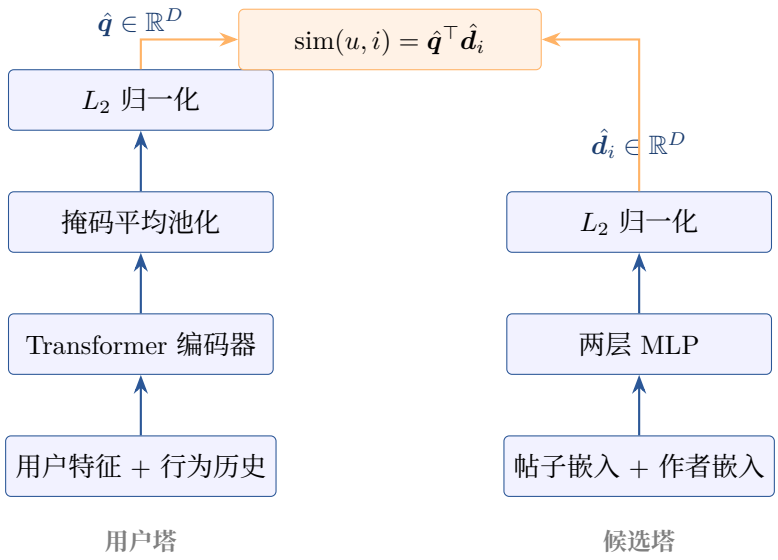

图 2: 双塔检索模型架构

3.2.1 模型架构

双塔模型由用户塔（User Tower）和候选塔（Candidate Tower）两个独立的编码器组成，如 fig. 2所示。

用户塔接收用户特征和行为历史，通过与排序模型共享的 Transformer 架构编码为一个归一化的用户嵌入向量；候选塔则将帖子特征通过两层 MLP 映射为归一化的候选嵌入向量。

3.2.2 检索流程

在线服务时，系统首先通过用户塔实时计算用户嵌入 $\hat{\boldsymbol{q}}$，然后利用点积相似度在预先构建的候选向量索引中进行近似最近邻（ANN）检索：

$$\mathcal{R} = \mathrm{TopK}_{i \in \mathrm{Corpus}} \{ \hat{\boldsymbol{q}}^\top \hat{\boldsymbol{d}}_i \} \tag{2}$$

由于候选塔的嵌入向量可以离线批量计算并建立索引，在线检索仅涉及一次用户塔推理和一次向量检索，延迟极低。

3.3 精排：Phoenix Ranking

精排是整个推荐系统中最核心的环节。Phoenix 排序模型基于 Grok 大语言模型的 Transformer 架构，针对推荐场景进行了专门适配。

3.3.1 模型输入

模型的输入序列由三部分拼接而成：

$$\boldsymbol{X} = [\underbrace{\boldsymbol{e}_u}_{1\text{ 位置}}; \underbrace{\boldsymbol{h}_1, \boldsymbol{h}_2, \ldots, \boldsymbol{h}_S}_{S\text{ 个历史位置}}; \underbrace{\boldsymbol{c}_1, \boldsymbol{c}_2, \ldots, \boldsymbol{c}_C}_{C\text{ 个候选位置}}] \in \mathbb{R}^{(1+S+C) \times D} \tag{3}$$

其中 $\boldsymbol{e}_u$ 为用户嵌入，$\boldsymbol{h}_s$ 为第 s 个历史交互的嵌入，$\boldsymbol{c}_j$ 为第 j 个候选帖子的嵌入。

3.3.2 候选隔离注意力

模型的核心创新在于**候选隔离注意力**机制。通过特殊的注意力掩码，确保候选帖子之间互不可见——每个候选只能关注用户上下文和自身，不能关注其他候选。详细数学描述见 section 5.2。

3.3.3 多行为预测

模型输出每个候选帖子在 $A = 19$ 种行为类型上的预测概率，涵盖正向行为（点赞、转发、回复、分享等）和负向行为（屏蔽、静音、举报等）。

3.4 重排阶段

重排阶段在精排得分基础上，进一步施加业务策略和多样性控制，包括四个子步骤：

(i) **加权融合**（Weighted Scorer）：将多行为概率线性加权为标量分数；

(ii) **作者多样性衰减**（Author Diversity Scorer）：对同一作者的多个帖子施加指数衰减；

(iii) **外网调整**（OON Scorer）：对外网候选施加缩放因子；

(iv) **可见性过滤**（VF Filter）：移除违规内容。

4 关键模型与算法

4.1 用户建模

系统中的用户表示来源于两个层面。

4.1.1 显式用户特征

显式用户特征（User Features）$\mathcal{U}$ 定义为以下集合的组合：

$$\mathcal{U} = \left(\mathcal{F}_{\text{follow}},\ \mathcal{F}_{\text{block}},\ \mathcal{F}_{\text{mute}},\ \mathcal{K}_{\text{muted}},\ \mathcal{F}_{\text{sub}}\right) \tag{4}$$

其中 $\mathcal{F}_{\text{follow}}$ 为关注列表，$\mathcal{F}_{\text{block}}$ 为屏蔽列表，$\mathcal{F}_{\text{mute}}$ 为静音列表，$\mathcal{K}_{\text{muted}}$ 为静音关键词集合，$\mathcal{F}_{\text{sub}}$ 为订阅列表。这些特征主要用于过滤阶段。

4.1.2 用户行为序列（UAS）

用户行为序列 $\mathcal{S} = \{(\text{post}_t, \text{author}_t, \boldsymbol{a}_t, \text{surface}_t)\}_{t=1}^{T}$ 记录了用户近期所有交互行为的有序集合，其中 $\boldsymbol{a}_t \in \{0,1\}^A$ 为第 t 个交互的 multi-hot 行为向量。UAS 经过以下预处理管道：

1) 保留原创行为并过滤衍生行为（`KeepOriginalUserActionFilter`）；

2) 聚合同一帖子上的多次行为（`DefaultAggregator`）；

3) 移除稀疏条目（`DenseAggregatedActionFilter`）；

4) 截断至最大序列长度 $S_{\max}$。

4.1.3 多哈希用户嵌入

用户嵌入通过多哈希嵌入查找（Multi-Hash Embedding Lookup）实现。设用户 ID 为 x，使用 H 个独立哈希函数 $\{f_h\}_{h=1}^{H}$ 和对应嵌入表 $\{\boldsymbol{E}_h \in \mathbb{R}^{V\times D}\}_{h=1}^{H}$，则：

$$\boldsymbol{e}_u = \mathrm{Concat}\big(\boldsymbol{E}_1[f_1(x)],\ \boldsymbol{E}_2[f_2(x)],\ \ldots,\ \boldsymbol{E}_H[f_H(x)]\big)\cdot \boldsymbol{W}_{\mathrm{proj}} \tag{5}$$

其中 $\boldsymbol{W}_{\mathrm{proj}} \in \mathbb{R}^{HD\times D}$ 为可学习的投影矩阵。

4.2 内容建模

帖子的表示同样基于多哈希嵌入。每个帖子通过帖子哈希和作者哈希两组嵌入来表示。

4.2.1 历史序列嵌入

对于行为历史中的第 t 个帖子，其嵌入为：

$$\boldsymbol{h}_t = \mathrm{Concat}\big(\boldsymbol{e}_{\mathrm{post},t},\ \boldsymbol{e}_{\mathrm{author},t},\ \boldsymbol{e}_{\mathrm{action},t},\ \boldsymbol{e}_{\mathrm{surface},t}\big)\cdot \boldsymbol{W}_{\mathrm{hist}} \tag{6}$$

其中各分量嵌入的计算方式为：

- $\boldsymbol{e}_{\mathrm{post},t}$：帖子 ID 的多哈希嵌入（拼接后展平）；
- $\boldsymbol{e}_{\mathrm{author},t}$：作者 ID 的多哈希嵌入（拼接后展平）；
- $\boldsymbol{e}_{\mathrm{action},t}$：行为嵌入，见 eq. (7)；
- $\boldsymbol{e}_{\mathrm{surface},t}$：产品界面嵌入（Embedding Table 查找）。

4.2.2 行为嵌入

行为嵌入采用符号化编码：

$$\boldsymbol{e}_{\mathrm{action}} = (2\boldsymbol{a} - \boldsymbol{1})\cdot \boldsymbol{W}_{\mathrm{act}} \odot \mathbb{1}[\|\boldsymbol{a}\|_1 > 0] \tag{7}$$

其中 $\boldsymbol{a} \in \{0,1\}^A$ 为 multi-hot 行为向量，$\boldsymbol{W}_{\mathrm{act}} \in \mathbb{R}^{A\times D}$ 为可学习的行为投影矩阵。符号化变换 $2\boldsymbol{a} - \boldsymbol{1}$ 将 $\{0,1\}$ 映射为 $\{-1,+1\}$，使得 “未发生” 的行为提供负信号。

4.2.3 候选帖子嵌入

候选帖子的嵌入与历史嵌入类似，但不包含行为信息（因为用户尚未与候选互动）：

$$\boldsymbol{c}_j = \mathrm{Concat}\big(\boldsymbol{e}_{\mathrm{post},j},\ \boldsymbol{e}_{\mathrm{author},j},\ \boldsymbol{e}_{\mathrm{surface},j}\big)\cdot \boldsymbol{W}_{\mathrm{cand}} \tag{8}$$

4.3 排序模型：Phoenix Transformer

Phoenix 排序模型的 Transformer 架构继承自 Grok-1 的核心设计，包含以下关键组件。

4.3.1 RMS 层归一化

代替传统的 LayerNorm，系统采用 RMSNorm：

$$\mathrm{RMSNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2 + \epsilon}} \odot \boldsymbol{\gamma} \tag{9}$$

其中 $\boldsymbol{\gamma}$ 为可学习的缩放参数，ϵ 为数值稳定性常数。RMSNorm 省略了均值中心化步骤，在保持训练稳定性的同时降低了计算开销。

4.3.2 旋转位置编码（RoPE）

RoPE 将位置信息编码为旋转变换。对于位置 m 处的查询向量 $\boldsymbol{q}$ 和位置 n 处的键向量 $\boldsymbol{k}$，旋转变换定义为：

$$\hat{\boldsymbol{q}}_m = \boldsymbol{R}(\boldsymbol{\theta}, m)\boldsymbol{q}_m, \qquad \hat{\boldsymbol{k}}_n = \boldsymbol{R}(\boldsymbol{\theta}, n)\boldsymbol{k}_n \tag{10}$$

其中旋转角度为：

$$\theta_j = \frac{1}{10000^{2j/d}}, \quad j = 0, 1, \ldots, \frac{d}{2} - 1 \tag{11}$$

旋转矩阵通过成对维度上的二维旋转实现：

$$\boldsymbol{R}(\boldsymbol{\theta}, m) = \begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & & & \\ \sin m\theta_0 & \cos m\theta_0 & & & \\ & & \cos m\theta_1 & -\sin m\theta_1 & \\ & & \sin m\theta_1 & \cos m\theta_1 & \\ & & & & \ddots \end{pmatrix} \tag{12}$$

性质 4.1 (相对位置性)**.** RoPE 的关键性质是注意力权重仅依赖于位置差：

$$\hat{\boldsymbol{q}}_m^\top \hat{\boldsymbol{k}}_n = \boldsymbol{q}_m^\top \boldsymbol{R}(\boldsymbol{\theta}, n-m)^\top \boldsymbol{k}_n \tag{13}$$

4.3.3 分组查询注意力（GQA）

系统允许 Query 头数量 n_q 大于 Key/Value 头数量 n_{kv}，其中 n_q/n_{kv} 个 Query 头共享同一组 Key/Value 头。这在保持模型表达力的同时显著减少了 KV Cache 的内存占用。

4.3.4 SwiGLU 前馈网络

前馈层采用 GELU 门控机制：

$$\mathrm{FFN}(\boldsymbol{x}) = \boldsymbol{W}_2\big(\mathrm{GELU}(\boldsymbol{W}_1\boldsymbol{x}) \odot \boldsymbol{V}\boldsymbol{x}\big) \tag{14}$$

其中 $\boldsymbol{W}_1, \boldsymbol{V} \in \mathbb{R}^{D_{\mathrm{ff}} \times D}$，$\boldsymbol{W}_2 \in \mathbb{R}^{D \times D_{\mathrm{ff}}}$。隐藏层维度 D_{ff} 按以下规则计算：

$$D_{\mathrm{ff}} = \left\lfloor \frac{2}{3} \cdot w \cdot D \right\rfloor + \left(8 - \left\lfloor \frac{2}{3} \cdot w \cdot D \right\rfloor \bmod 8\right) \tag{15}$$

其中 w 为宽度因子（widening factor），取 8 的倍数以优化硬件利用率。

4.3.5 注意力 Logit 裁剪

在 Softmax 之前，对注意力 logits 施加 tanh 裁剪：

$$\text{logits}_{\text{clipped}} = \alpha \cdot \tanh\left(\frac{\text{logits}}{\alpha}\right), \quad \alpha = 30.0 \tag{16}$$

这将注意力 logits 限制在 $[-\alpha, \alpha]$ 范围内，防止训练过程中出现极端的注意力权重。

4.3.6 Decoder Layer 结构

每个 Decoder Layer 遵循 Pre-Norm 残差连接结构：

$$\boldsymbol{h}' = \boldsymbol{h} + \text{RMSNorm}\big(\text{MHA}(\text{RMSNorm}(\boldsymbol{h}), \boldsymbol{M})\big) \tag{17}$$

$$\boldsymbol{h}'' = \boldsymbol{h}' + \text{RMSNorm}\big(\text{FFN}(\text{RMSNorm}(\boldsymbol{h}'))\big) \tag{18}$$

其中 $\boldsymbol{M}$ 为注意力掩码矩阵，MHA 为多头注意力，FFN 为 SwiGLU 前馈网络。

4.4 特征工程方法

X 推荐系统做出了一个激进的设计选择：**完全消除手工特征工程**。系统不依赖任何人工设计的内容相关性特征（如 TF-IDF、话题标签匹配度、发帖时间衰减函数等），而是将所有特征学习的任务交给端到端的 Transformer 模型。

模型的输入特征仅包含四类原始信号：

(1) 实体 ID 的多哈希嵌入（用户、帖子、作者）；

(2) 用户行为的 multi-hot 编码；

(3) 产品界面的类别编码；

(4) 序列位置信息（通过 RoPE 隐式编码）。

这种 “去特征工程化” 的策略大幅简化了数据管道和服务基础设施的复杂度，但对模型的表达能力和训练数据的规模提出了更高的要求。

4.5 过滤策略体系

系统在评分前后分别设置了两组过滤器，形成了双层过滤体系，如 table 3所示。

值得注意的是 `PreviouslySeenPostsFilter` 采用**布隆过滤器**（Bloom Filter）实现已浏览帖子的高效判定。客户端在每次请求时将已见帖子的布隆过滤器编码发送至服务端，以极低的内存开销实现近似精确的去重。

5 数学原理与公式

5.1 双塔模型与向量检索

双塔检索模型的数学基础是度量学习（Metric Learning）。

表 3: 过滤器体系

过滤器	阶段	策略描述
DropDuplicatesFilter	评分前	基于帖子 ID 去重
CoreDataHydrationFilter	评分前	移除元数据加载失败的帖子
AgeFilter	评分前	移除超过保留期的旧帖子
SelfTweetFilter	评分前	移除用户自己的帖子
RetweetDeduplicationFilter	评分前	对同一内容的多次转发去重
IneligibleSubscriptionFilter	评分前	移除用户未订阅的付费内容
PreviouslySeenPostsFilter	评分前	通过布隆过滤器移除已浏览帖子
MutedKeywordFilter	评分前	基于 Token 序列匹配静音关键词
AuthorSocialgraphFilter	评分前	移除已屏蔽/静音作者的帖子
VFFilter	评分后	可见性过滤（删除/垃圾/暴力等）
DedupConversationFilter	评分后	每个会话树仅保留最高分候选

定义 5.1 (用户塔编码)**.** 设用户嵌入与行为历史嵌入的拼接序列为 $\boldsymbol{X}_u = [\boldsymbol{e}_u; \boldsymbol{h}_1, \ldots, \boldsymbol{h}_S]$，用户塔的编码过程为：

$$\boldsymbol{Z}_u = \text{Transformer}(\boldsymbol{X}_u) \in \mathbb{R}^{(1+S)\times D} \tag{19}$$

$$\bar{\boldsymbol{z}}_u = \frac{\sum_{t=1}^{1+S} m_t \cdot \boldsymbol{Z}_u[t]}{\sum_{t=1}^{1+S} m_t} \quad \text{(掩码平均池化)} \tag{20}$$

$$\hat{\boldsymbol{q}} = \frac{\bar{\boldsymbol{z}}_u}{\|\bar{\boldsymbol{z}}_u\|_2} \in \mathbb{R}^D \tag{21}$$

其中 $m_t \in \{0, 1\}$ 为 padding 掩码。

定义 5.2 (候选塔编码)**.** 候选塔将帖子嵌入与作者嵌入的拼接通过两层 MLP 投影并归一化：

$$\boldsymbol{v}_i = \text{Concat}(\boldsymbol{e}_{\text{post},i},\ \boldsymbol{e}_{\text{author},i}) \tag{22}$$

$$\hat{\boldsymbol{d}}_i = \frac{\boldsymbol{W}_2 \cdot \text{SiLU}(\boldsymbol{W}_1 \cdot \boldsymbol{v}_i)}{\|\boldsymbol{W}_2 \cdot \text{SiLU}(\boldsymbol{W}_1 \cdot \boldsymbol{v}_i)\|_2} \in \mathbb{R}^D \tag{23}$$

由于嵌入向量已经 L_2 归一化，点积相似度等价于余弦相似度：

$$\text{sim}(u, i) = \hat{\boldsymbol{q}}^\top \hat{\boldsymbol{d}}_i = \cos(\hat{\boldsymbol{q}}, \hat{\boldsymbol{d}}_i) \tag{24}$$

检索通过 Top-K 操作实现：

$$\mathcal{R} = \underset{i \in \text{Corpus},\ |\mathcal{R}|=K}{\arg\max} \ \hat{\boldsymbol{q}}^\top \hat{\boldsymbol{d}}_i \tag{25}$$

在实际系统中，候选塔的嵌入向量离线计算并建立 ANN 索引，在线检索的时间复杂度近似为 $O(\log N)$。

5.2 候选隔离注意力机制

候选隔离是 Phoenix 排序模型最核心的设计决策。

定义 5.3 (候选隔离注意力掩码). 设总序列长度为 $L = 1 + S + C$，候选起始偏移为 $\delta = 1 + S$。注意力掩码矩阵 $\boldsymbol{M} \in \mathbb{R}^{L \times L}$ 定义如下：

$$M_{ij} = \begin{cases} \mathbb{1}[j \le i] & \text{if } i < \delta \text{ and } j < \delta \quad \text{(因果注意力)} \\ 0 & \text{if } i < \delta \text{ and } j \ge \delta \quad \text{(历史不看候选)} \\ 1 & \text{if } i \ge \delta \text{ and } j < \delta \quad \text{(候选看历史)} \\ \mathbb{1}[i = j] & \text{if } i \ge \delta \text{ and } j \ge \delta \quad \text{(候选仅自注意力)} \end{cases} \tag{26}$$

以一个具体的 6 位置序列为例（$S = 2, C = 3$），掩码矩阵为：

$$\boldsymbol{M} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & \mathbf{1} & 0 & 0 \\ 1 & 1 & 1 & 0 & \mathbf{1} & 0 \\ 1 & 1 & 1 & 0 & 0 & \mathbf{1} \end{pmatrix} \tag{27}$$

性质 5.1 (评分独立性). 由于掩码矩阵的结构，候选 c_j 的 Transformer 输出 $\boldsymbol{z}_j$ 仅依赖于用户上下文 $\{\boldsymbol{e}_u, \boldsymbol{h}_1, \ldots, \boldsymbol{h}_S\}$ 和自身输入 $\boldsymbol{c}_j$：

$$\boldsymbol{z}_j = f_{\text{Transformer}}(\boldsymbol{e}_u, \boldsymbol{h}_1, \ldots, \boldsymbol{h}_S, \boldsymbol{c}_j; \boldsymbol{\theta}) \tag{28}$$

与批次中其他候选 $\{c_k\}_{k \ne j}$ 无关。这保证了评分的一致性与可缓存性。

5.3 多行为联合预测

Phoenix 模型的输出层将每个候选位置的 Transformer 隐藏表示通过反嵌入矩阵投影到行为空间：

$$\boldsymbol{\ell}_j = \text{RMSNorm}(\boldsymbol{z}_j) \cdot \boldsymbol{W}_{\text{unembed}} \in \mathbb{R}^A \tag{29}$$

其中 $\boldsymbol{z}_j \in \mathbb{R}^D$ 是候选 j 的 Transformer 输出，$\boldsymbol{W}_{\text{unembed}} \in \mathbb{R}^{D \times A}$ 是反嵌入矩阵，$A = 19$。

概率通过 Sigmoid 函数**独立**计算（而非 Softmax），因为各行为之间并非互斥：

$$P(\text{action}_a \mid u, j) = \sigma(\ell_{j,a}) = \frac{1}{1 + e^{-\ell_{j,a}}}, \quad a = 1, 2, \ldots, A \tag{30}$$

系统预测的 19 种行为类型如 table 4所示。

5.4 加权评分公式

Weighted Scorer 将多行为概率融合为一个标量分数：

$$s_{\text{raw}}(j) = \sum_{a=1}^{A} w_a \cdot P(\text{action}_a \mid u, j) \tag{31}$$

其中 w_a 为每种行为的权重。正向行为赋予正权重（$w_a > 0$），负向行为赋予负权重（$w_a < 0$）。

表 4: 多行为预测的 19 种行为类型

编号	行为	编号	行为
1	Favorite（点赞）	11	Dwell（停留）
2	Reply（回复）	12	Quote（引用）
3	Repost（转发）	13	Quoted Click
4	Photo Expand	14	Follow Author
5	Click（点击）	15	Not Interested（负）
6	Profile Click	16	Block Author（负）
7	Video Quality View	17	Mute Author（负）
8	Share（分享）	18	Report（负）
9	Share via DM	19	Dwell Time（连续值）
10	Share via Copy Link		

5.4.1 分数归一化

为确保最终分数非负且具有可比性，系统采用偏移策略：

$$s(j) = \begin{cases} s_{\text{raw}}(j) + \Delta & \text{if } s_{\text{raw}}(j) \geq 0 \\ \dfrac{s_{\text{raw}}(j) + |W^-|}{|W|} \cdot \Delta & \text{if } s_{\text{raw}}(j) < 0 \end{cases} \tag{32}$$

其中：

$$|W^-| = \sum_{a:w_a<0} |w_a| \quad \text{(负权重绝对值之和)} \tag{33}$$

$$|W| = \sum_{a=1}^{A} |w_a| \quad \text{(所有权重绝对值之和)} \tag{34}$$

$$\Delta \geq 0 \quad \text{(非负偏移常数)} \tag{35}$$

这种设计确保负分数被压缩到 $[0, \Delta]$ 区间内，正分数在 $[\Delta, +\infty)$ 区间内。

5.4.2 视频权重条件

对于视频内容，VQV（Video Quality View）行为的权重取决于视频时长：

$$w_{\text{vqv}}(j) = \begin{cases} w^*_{\text{vqv}} & \text{if duration}(j) > T_{\min} \\ 0 & \text{otherwise} \end{cases} \tag{36}$$

其中 $T_{\min}$ 为最小视频时长阈值。

5.5 作者多样性衰减

Author Diversity Scorer 基于指数衰减函数控制同一作者的帖子密度。

定义 5.4 (多样性乘法因子). 设某作者在候选集中按加权分数排序后，其第 k 个帖子（k 从 0 开始）的乘法因子为:

$$\mu(k) = (1-\phi)\cdot\lambda^k + \phi \tag{37}$$

其中 $\lambda \in (0,1)$ 为衰减因子，$\phi \in (0,1)$ 为下限系数（floor）。

该作者第 k 个帖子的最终分数为:

$$s_{\text{final}}(k) = s_{\text{weighted}} \cdot \mu(k) \tag{38}$$

性质 5.2 (衰减特性).

- 当 $k = 0$ 时，$\mu(0) = 1$（第一个帖子不受惩罚）;
- 当 $k \to \infty$ 时，$\mu(k) \to \phi$（乘法因子趋向下限）;
- 衰减速率由 λ 控制，λ 越小衰减越快。

注记 5.1. 算法在实现时首先按加权分数全局降序排序所有候选，然后按序处理。处理每个候选时，查找该作者已出现的次数 k，计算 $\mu(k)$ 并更新分数。这确保了高分帖子优先获得较少的惩罚。

5.6 外网内容调整

OON Scorer 对来自用户未关注账户的帖子施加缩放:

$$s_{\text{oon}}(j) = \begin{cases} s(j)\cdot\beta_{\text{oon}} & \text{if } j \notin \text{InNetwork} \\ s(j) & \text{otherwise} \end{cases} \tag{39}$$

其中 $\beta_{\text{oon}} \in (0,1]$ 为外网缩放因子。

5.7 哈希嵌入与冲突缓解

定义 5.5 (多哈希嵌入). 设实体 ID 为 x，使用 H 个独立哈希函数 $\{f_h : \mathbb{Z} \to [V]\}_{h=1}^{H}$ 和嵌入表 $\{\boldsymbol{E}_h \in \mathbb{R}^{V\times D}\}_{h=1}^{H}$:

$$\boldsymbol{e}(x) = \underbrace{\left[\boldsymbol{E}_1[f_1(x)] \,\|\, \boldsymbol{E}_2[f_2(x)] \,\|\, \cdots \,\|\, \boldsymbol{E}_H[f_H(x)]\right]}_{H\text{ 个嵌入拼接}} \cdot \boldsymbol{W}_{\text{proj}} \tag{40}$$

其中 $\boldsymbol{W}_{\text{proj}} \in \mathbb{R}^{HD\times D}$ 为可学习投影矩阵，$\|$ 表示向量拼接。

性质 5.3 (冲突率分析). 设单个哈希函数的冲突率为 $p = 1/V$（理想均匀哈希），则 H 个独立哈希函数**同时冲突**的概率为:

$$P(\text{all collide}) = p^H = \frac{1}{V^H} \tag{41}$$

对于 $V = 10^5$，$H = 2$，实际冲突率约为 10^{-10}，可以忽略不计。

5.8 布隆过滤器在去重中的应用

`PreviouslySeenPostsFilter` 使用布隆过滤器实现已浏览帖子的概率去重。

定义 5.6 (布隆过滤器)**.** 布隆过滤器 $\mathcal{B}$ 由一个长度为 m 的位数组与 k 个独立哈希函数组成，记作 $\{g_1, \ldots, g_k\}$。对于元素 x：

- **插入**：将位数组中 $g_1(x), g_2(x), \ldots, g_k(x)$ 位置设为 1；
- **查询**：若所有 $g_1(x), g_2(x), \ldots, g_k(x)$ 位置均为 1，则返回" 可能存在"，否则" 确定不存在"。

对于包含 n 个元素的布隆过滤器，假阳性率为：

$$P(\text{false positive}) \approx \left(1 - e^{-kn/m}\right)^k \tag{42}$$

客户端在每次请求时将已见帖子的布隆过滤器编码发送至服务端，服务端据此高效判断候选帖子是否已被浏览，实现了在带宽与精度之间的最优权衡。

6 管道框架设计

Candidate Pipeline 框架是系统架构的基石，它将推荐管道抽象为一组可组合的阶段。

6.1 Trait 抽象

框架定义了六个核心 Trait，如 table 5所示。

表 5: Candidate Pipeline 核心 Trait

Trait	用途	执行模式
`Source`	获取候选	多 Source 并行执行
`Hydrator`	丰富候选特征	多 Hydrator 并行执行
`Filter`	移除不合格候选	多 Filter 串行执行
`Scorer`	计算候选分数	多 Scorer 串行执行
`Selector`	排序并截断	单一 Selector
`SideEffect`	异步副作用	多 SideEffect 并行异步执行

6.2 设计原则

- **关注点分离**：管道的执行逻辑与业务逻辑完全解耦。新增一个过滤器或评分器只需实现对应的 Trait。
- **并行与串行的精确控制**：Sources 和 Hydrators 并行执行（因为相互独立），Filters 和 Scorers 串行执行（因为可能依赖前序结果），Side Effects 在结果返回后异步并行执行。
- **优雅的错误处理**：每个组件的失败不导致管道崩溃。Source 失败时忽略该来源，Hydrator 失败时保持候选不变，Filter 失败时保留所有候选，Scorer 失败时保持分数不变。

- **长度校验机制**：Hydrators 和 Scorers 返回的结果长度必须与输入严格匹配，否则跳过并记录警告。

7 设计哲学与讨论

7.1 去特征工程化

X 推荐系统最引人注目的设计选择是完全放弃手工特征工程。传统推荐系统依赖大量人工设计的特征——热度指标、话题匹配度、时间衰减函数、用户画像标签等——来辅助排序。而 X 的系统将所有这些 “先验知识” 交给 Transformer 模型自动学习。

这一决策的理论依据在于：大规模 Transformer 模型具有足够的表达能力来发现数据中的复杂模式。与其花费大量人力设计和维护特征管道（特征漂移、计算延迟、交叉组合爆炸），不如投入更多计算资源训练更大的模型。

7.2 候选隔离的工程意义

候选隔离注意力不仅是算法创新，更是深思熟虑的工程决策。在没有隔离的情况下，候选 A 的得分会因同一批次中候选 B 的存在而变化，导致：

(i) 评分不可复现（不同批次组合产生不同分数）；

(ii) 评分不可缓存（无法预先计算单个帖子的分数）。

候选隔离通过 eq. (26) 中注意力掩码的简单修改，在不牺牲模型表达力的前提下优雅地解决了这两个问题。

7.3 正负信号的联合建模

系统同时预测 “正向” 和 “负向” 行为的设计反映了对推荐质量的深层理解。考虑 eq. (31) 中的加权评分公式，一个高点赞概率但同时高举报概率的帖子：

$$s = w_{\text{fav}} \cdot P(\text{fav}) + w_{\text{report}} \cdot P(\text{report}) + \cdots \tag{43}$$

由于 $w_{\text{report}} < 0$，其综合价值远低于仅考虑 $P(\text{fav})$ 时的估计。这种联合建模捕获了帖子的 “净效用”——正面体验减去负面体验的总和。

7.4 内外网融合策略

系统通过 OON Scorer 的缩放因子 β_{oon} 来平衡内网与外网内容的比例。这一机制说明：纯粹基于分数排序并不能保证最优的用户体验。用户期望同时满足社交属性（来自关注账户的内容）和媒体属性（新的有趣内容）。β_{oon} 本质上是在这两种需求之间寻找平衡的调节参数。

8 总结

本文系统分析了 X 平台 For You 推荐系统的技术架构与算法原理。该系统的核心设计理念可概括为以下五点：

(1) **端到端学习优先**：通过 Grok-based Transformer 模型实现端到端的行为预测，消除手工特征工程，将系统复杂度从数据管道转移到模型训练环节。

(2) **多阶段漏斗架构**：通过召回（百万级 → 千级）、精排（千级 → 百级）、重排（百级 → 几十级）的逐步筛选，在保证覆盖率的同时实现高精度排序。

(3) **评分一致性保证**：通过候选隔离注意力机制（eq. (26)）确保评分的批次无关性，提升系统的可预测性和可调试性。

(4) **正负信号联合建模**：通过多行为联合预测（eq. (30)）和加权融合（eq. (31)），实现对用户“净效用”的精确估计，超越了传统单一 CTR 预估的范式。

(5) **可组合的管道框架**：通过 Trait 抽象和管道编排机制，实现了推荐系统各组件的高度模块化和可扩展性。

从学术视角看，X 的推荐系统代表了“大模型驱动推荐”范式的前沿实践。与传统的“特征工程 + 浅层模型”路线相比，该系统以更简洁的数据管道和更强大的模型表达力来应对推荐问题的复杂性。随着 Transformer 模型在推荐场景中的进一步发展，这种“去特征化”的设计理念有望成为下一代推荐系统的主流方向。

A 符号表

表 6: 主要数学符号汇总

符号	含义	符号	含义
D	嵌入维度	A	行为类型数（19）
S	历史序列长度	C	候选数量
L	总序列长度 $1+S+C$	δ	候选起始偏移 $1+S$
H	哈希函数数量	V	嵌入表词表大小
$\hat{\boldsymbol{q}}$	归一化用户嵌入	$\hat{\boldsymbol{d}}_i$	归一化候选嵌入
$\boldsymbol{M}$	注意力掩码矩阵	$\boldsymbol{W}_{\text{unembed}}$	反嵌入矩阵
w_a	行为 a 的权重	$\sigma(\cdot)$	Sigmoid 函数
λ	作者多样性衰减因子	ϕ	多样性下限系数
β_{oon}	外网缩放因子	Δ	分数偏移常数
$\boldsymbol{\theta}$	模型参数	$\boldsymbol{\gamma}$	RMSNorm 缩放参数